CSS4003 — DevOps 2

Lecture 11: CI/CD with GitHub Actions

Nauruzbayeva Farikha · Lesson 11

Recap — Lesson 10 (Docker in Production)

- Tag vs digest? (a tag is a movable name; a digest is the content hash — same digest, same image, forever)
- Why does the server only pull, never build? (it must run the exact image that was tested)
- What did Trivy do for us? (scanned the image for known CVEs and failed the gate on HIGH/CRITICAL)
- How did we roll back? (changed the tag back to v1 and `docker compose up -d` — the old image was still in the registry)
- What does an orchestrator add? (scheduling, self-healing, scaling, rolling updates)

Today’s agenda

- ☐ CI, continuous delivery, continuous deployment
- ☐ GitHub Actions anatomy: events, jobs, steps, runners, actions
- ☐ One workflow, step by step: lint, tests, matrix, services, artifacts, cache
- ☐ Build and push the image to GHCR with meaningful tags
- ☐ Security gates: gitleaks, Trivy, pip-audit, least-privilege tokens
- ☐ Branch protection: red check = no merge
- ☐ CD: approvals, deploy over SSH, rollback
- ☐ Reuse, self-hosted runners, debugging
- ☐ Live demo → straight into today’s practice

The scenario

Last week **you** built the image, ran Trivy, pushed v2 and edited `.env` on the server. It worked — on a good day. On a bad day someone forgets the tests, pushes from a laptop with uncommitted changes, or tags `latest` by mistake.

Make every change go through the same steps, automatically: test, scan, build, push, deploy — and refuse anything that fails.

Block 1

CI and CD

Three words, three promises

Term	Promise	Ends with
Continuous Integration	Every change is built and tested together, many times a day	A green or red check
Continuous Delivery	Every green change *can* be released at any time	A deployable artifact (image)
Continuous Deployment	Every green change *is* released automatically	Running in production

Most teams do CI + continuous delivery, with a **human approval** before production. That's what we build today.

Why bother

- **Fast feedback** — a broken test is found in minutes, by a machine, not by a user next week.
- **Same steps every time** — no "I forgot to run the linter".
- **Small changes** — merging often keeps conflicts small.
- **Traceability** — every image and deploy points to a commit and a run.
- **Safe releases** — rollback is "deploy the previous tag", not heroics.

> The pipeline is the **only** way code reaches production. If it's not in git and green, it doesn't ship.

The pipeline we'll build

```
pull request ─► lint ─► test (3.11, 3.12) ─► secret scan ─► ✅ / ❌  (merge allowed?)
push to main ─► … same … ─► build ─► Trivy ─► push ghcr.io/…:sha-…, main
tag v1.2.0   ─► … same … ─► build ─► Trivy ─► push ghcr.io/…:1.2.0
manual       ─► approve ─► deploy 1.2.0 over SSH ─► (or roll back to 1.1.0)
```

One YAML file for CI and image, one for deploy — both live in the repo, reviewed like any other code.

Block 2

GitHub Actions anatomy

Workflow → jobs → steps

```
.github/workflows/ci.yml        ← a workflow (YAML file)
  on: pull_request, push        ← events that start it
  jobs:
    lint:                        ← a job: runs on its own fresh machine (runner)
      steps:                     ← steps run in order, in the same machine
        - uses: actions/checkout@v7      ← a reusable action
        - run: ruff check .               ← a shell command
    test:                        ← jobs run in parallel unless you add needs:
```

- **Workflow** — a YAML file in `.github/workflows/`.
- **Job** — a set of steps on one **runner** (VM). Jobs don't share files.
- **Step** — either `uses:` an action or `run:` a shell command.

Events: when does it run?

```yaml
on:
  pull_request:                # every PR (and every push to it)
  push:
    branches: [main]           # merges to main
    tags: ["v*"]               # release tags like v1.2.0
  workflow_dispatch:           # a "Run workflow" button (with inputs)
  schedule:
    - cron: "0 3 * * 1"        # every Monday 03:00 UTC (e.g. a weekly scan)
  release:
    types: [published]         # when a GitHub Release is published
```

Filters (`branches` , `tags` , `paths`) keep pipelines from running for nothing — e.g. skip CI when only `docs/` changed.

Runners

- **GitHub-hosted** — a fresh VM per job: `ubuntu-latest`, `windows-latest`, `macos-latest`. Docker, git, Python, Node are pre-installed. Thrown away after the job.
- Free minutes for public repositories; private repos get a monthly quota.
- **Self-hosted** — your own machine runs the jobs (Block 7).
- Every job starts **empty**: no files from the previous job, no dependencies — that's why we cache and use artifacts.

Actions: reuse, but pin

```yaml
- uses: actions/checkout@v7                      # major version tag
- uses: actions/setup-python@v7
- uses: docker/build-push-action@v7
- uses: some-org/some-action@8f4b7f84bd579b95d7f0b90f8d8b6e5d9b4f1d3a  # full SHA
```

- An action is code from someone else's repo that runs **with your token and secrets**.
- `@v7` follows the latest v7.x — convenient, but the owner can move it.
- `@<full commit SHA>` can never change — the safest choice for third-party actions.
- Prefer actions from `actions/`, `docker/` and well-known vendors.

Quick check

The `test` job needs the Docker image built by the `build` job. Will it find it?

No — every job runs on a fresh runner. Pass files with **artifacts** (or push the image to a registry), and order jobs with `needs:`.

Block 3

One workflow, step by step

Start: trigger, permissions, concurrency

```yaml
name: ci

on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*"]

permissions:
  contents: read            # least privilege for every job

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true  # a new push cancels the old run of that branch
```

`${{ … }}` is an **expression**: GitHub fills it in before the job starts (`github.ref` = the branch or tag).

Job 1: lint

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: ruff format --check .
```

- `checkout` clones your repo into the runner — without it the runner is empty.
- `cache: pip` restores downloaded packages from the last run: faster installs.
- Any command that exits non-zero fails the step → the job → the check.

Job 2: tests on two Python versions

```yaml
test:
  runs-on: ubuntu-latest
  strategy:
    matrix:
      python-version: ["3.11", "3.12"]
  steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-python@v7
      with: { python-version: "${{ matrix.python-version }}", cache: pip }
    - run: pip install -r requirements-dev.txt
    - run: pytest -q --junitxml=report-${{ matrix.python-version }}.xml
```

- The **matrix** creates one job per value: `test (3.11)` and `test (3.12)`, running in parallel.
- Add more axes (OS, DB version) → every combination becomes a job.

A real database in CI: service containers

```yaml
test:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:17
      env:
        POSTGRES_PASSWORD: test        # throwaway, CI-only
      ports: ["5432:5432"]
      options: >-
        --health-cmd "pg_isready -U postgres"
        --health-interval 5s --health-retries 10
  steps:
    - uses: actions/checkout@v7
    - run: pytest -q
      env: { DB_URL: "postgresql://postgres:test@localhost:5432/postgres" }
```

The runner starts Postgres **before** the steps and waits until the health check passes. Integration tests hit a real DB, then it's thrown away.

Artifacts and test reports

```yaml
      - uses: actions/upload-artifact@v7
        if: always()              # upload even when tests failed
        with:
          name: test-report-${{ matrix.python-version }}
          path: report-${{ matrix.python-version }}.xml
```

- **Artifacts** — files saved from a run: test reports, coverage, built binaries. Download them from the run's Summary page.
- Another job can fetch them with `actions/download-artifact`.
- `if: always()` — a failed run is exactly when you want the report.
- Artifacts expire (90 days by default) — they're not a registry.

Cache vs artifact

	Cache	Artifact
Purpose	Speed up the **next** run	Keep **this** run's output
Example	pip downloads, Docker layers	Test report, coverage, binary
Key	Hash of `requirements*.txt`	Name you choose
Missing?	Run is slower, still correct	Something downstream breaks

A cache must never change the result — only the time. If a stale cache breaks your build, the cache key is wrong.

Ordering jobs with `needs` and `if`

```
  image:
    needs: [lint, test, secrets-scan]   # wait for all three, all must be green
    if: github.event_name == 'push'      # not on pull requests
    runs-on: ubuntu-latest
```

- Without `needs:` jobs run in **parallel**.
- `needs:` = "start after these succeeded" — one red job stops the rest.
- `if:` skips a job (shown as grey "skipped"): here we build and push images only for `main` and tags, never for PRs.

Block 4

Build and push the image

Logging in to GHCR from Actions

```yaml
permissions:
  contents: read
  packages: write          # only this job may push packages
steps:
  - uses: actions/checkout@v7
  - uses: docker/setup-buildx-action@v4
  - uses: docker/login-action@v4
    with:
      registry: ghcr.io
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}
```

- `GITHUB_TOKEN` is created for each run and **expires** when the job ends — no personal token to leak.
- What it can do is set by `permissions:` — here: read code, write packages.

Tags without thinking: `metadata-action`

```yaml
      - id: meta
        uses: docker/metadata-action@v6
        with:
          images: ghcr.io/farikha/lab11-cicd
          tags: |
            type=sha                          # sha-3f9c2e1
            type=semver,pattern={{version}}   # 1.2.0  (from tag v1.2.0)
            type=ref,event=branch             # main
```

Event	Tags pushed
push to `main`	`main` , `sha-3f9c2e1`
tag `v1.2.0`	`1.2.0` , `sha-3f9c2e1`

GHCR image names must be **lower case** — lower-case `github.repository` before using it as an image name.

Build, scan, then push

```yaml
      - uses: docker/build-push-action@v7       # 1. build, keep it local
        with:
          context: .
          load: true
          tags: lab11-app:scan
          cache-from: type=gha
      - uses: aquasecurity/trivy-action@v0.36.0  # 2. scan the local image
        with:
          image-ref: lab11-app:scan
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"
      - uses: docker/build-push-action@v7       # 3. push only if the scan passed
        with:
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
```

Why this order?

- A vulnerable image is **never pushed** — the gate fails before step 3.
- `cache-from: type=gha` stores Docker layers in the Actions cache: the second build is mostly cache hits.
- The push re-uses the same layers — it doesn’t really "build twice".
- `labels` add OCI metadata: source repo, commit, created date — `docker inspect` shows which commit an image came from.

Quick check

Your `image` job fails with `denied: permission_denied: write_package`. The login step was green. What's missing?

`permissions: packages: write` on that job. Logging in only proves who you are; the token's permissions decide what you may do.

Block 5

Security gates

Gates: a check that can say "no"

Gate	Tool	Fails when
Lint / format	ruff	Style or obvious bugs
Tests	pytest	Behaviour broke
Secrets in git	gitleaks	A token or key is in any commit
Image CVEs	Trivy	HIGH/CRITICAL with a fix available
Dependency CVEs	pip-audit	A pinned package is known-vulnerable
IaC misconfig	`trivy config`	Public buckets, open ports… (next week)

A gate is only a gate if it's **required** by branch protection and nobody switches it off "to make it pass".

gitleaks and pip-audit

```
  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
        with: { fetch-depth: 0 }      # the whole history, not just HEAD
      - run: |
          docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:v8.30.1 \
            detect --source /repo --redact --exit-code 1
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: pipx run pip-audit -r requirements.txt
```

A secret removed in a later commit is still in history — that's why gitleaks scans all commits. Found one? **Rotate it first**, then clean history.

Secrets, variables, environments

	Visible in logs?	Example	Where
Secret	Masked `***`	SSH key, API token	Settings → Secrets
Variable	Yes	`APP_PORT=8080`	Settings → Variables
Environment secret	Masked	Prod SSH key	Only jobs with `environment: production`

- `${{ secrets.NAME }}` — not available to PRs from **forks** (on purpose).
- Never `echo` a secret, never write it to a file that becomes an artifact.
- Masking is a safety net, not a guarantee: a secret printed base64 is not masked.

Least privilege for `GITHUB_TOKEN`

```yaml
permissions:
  contents: read          # workflow default: read the code, nothing else

jobs:
  image:
    permissions:
      contents: read
      packages: write     # only the job that pushes
```

- With no `permissions:` the token may get broad write access (depends on repo settings).
- A compromised step can only do what the token allows.
- Declare permissions per job: each job gets exactly what it needs.

OIDC: no stored cloud keys at all

- The classic way: put a long-lived cloud key in `secrets.` — it can leak, and it never expires.
- **OIDC**: the job asks GitHub for a short-lived **identity token** (`permissions: id-token: write`), the cloud provider trusts GitHub and swaps it for temporary credentials.
- The trust rule can say "only this repo, only branch `main`, only environment `production`".
- Every major cloud and tools like Vault support it — you'll see it again in your final project.

Block 6

Protect `main`, then deploy

Branch protection (rulesets)

- **Settings → Rules → Rulesets** (or *Branches* in the older UI) on `main`:
- Require a **pull request** before merging (and N approvals in a team).
- Require **status checks**: `lint`, `test (3.11)`, `test (3.12)`, `secrets-scan`.
- Block force pushes and deletions.
- Result: a red check = the **Merge** button is disabled. Nobody pushes straight to `main`, not even you.

Environments and approvals

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production     # needs a reviewer, unlocks prod secrets
```

- **Settings → Environments → production → Required reviewers**.
- The job **pauses** until a reviewer clicks *Approve* — that's continuous **delivery** with a human gate.
- Environment secrets (prod SSH key) are only given to jobs that name that environment.
- The environment page shows **which commit is deployed** and when.

Deploy over SSH

```yaml
on:
  workflow_dispatch:
    inputs:
      tag: { description: "Image tag to deploy", required: true }
concurrency: { group: deploy-production, cancel-in-progress: false }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: appleboy/ssh-action@v1.2.5
        env: { TAG: "${{ inputs.tag }}" }
        with:
          host: ${{ secrets.DEPLOY_HOST }}
          username: ${{ secrets.DEPLOY_USER }}
          key: ${{ secrets.DEPLOY_SSH_KEY }}
          envs: TAG
```

…and what runs on the server

```
set -euo pipefail
cd ~/lab11
sed -i "s/^TAG=.*/TAG=${TAG}/" .env
docker compose pull
docker compose up -d
docker compose ps
```

- Exactly last week's manual release — now in git, reviewed, logged.
- `concurrency` with `cancel-in-progress: false`: two deploys never run at once, and a running deploy is never cut in half.
- A dedicated **deploy key**, only for this server, only in the environment secrets.

Rollback = deploy an older tag

- Every release is an immutable image in the registry: `1.0.0`, `1.0.1`, `1.1.0`…
- Rollback: **Run workflow** with `1.0.0` → approve → done in seconds.
- Nothing is rebuilt — you deploy the exact image that ran before.
- Database migrations are the hard part: make them backwards-compatible so the old version still works with the new schema.

> Practise the rollback **before** you need it. An untested rollback is just a hope.

Quick check

v1.1.0 is broken in production. Do you revert the commit, wait for the pipeline and deploy v1.1.1 — or deploy 1.0.1 right now?

Roll back first (deploy 1.0.1: seconds), then fix forward calmly with a reviewed PR and a new tag. Restore service, then investigate.

Block 7

Reuse, runners, debugging

Reusable workflows

```yaml
# .github/workflows/python-ci.yml  (called by others)
on:
  workflow_call:
    inputs:
      python-version: { type: string, default: "3.12" }
```

```yaml
# .github/workflows/ci.yml  (the caller)
jobs:
  ci:
    uses: my-org/pipelines/.github/workflows/python-ci.yml@v1
    with: { python-version: "3.12" }
```

One pipeline definition, many repositories: fix it once, every team gets the fix.

Composite actions

```yaml
# .github/actions/setup-app/action.yml
name: setup-app
runs:
  using: composite
  steps:
    - uses: actions/setup-python@v7
      with: { python-version: "3.12", cache: pip }
    - run: pip install -r requirements-dev.txt
      shell: bash
```

```yaml
    - uses: ./.github/actions/setup-app     # in any job of this repo
```

Reusable workflow = a whole job (or jobs). Composite action = a few steps you repeat inside jobs.

Self-hosted runners

- **When:** you need special hardware, access to a private network (a server behind a firewall), or more minutes than you have.
- **How:** Settings → Actions → Runners → New, run the agent; `runs-on: self-hosted`.
- **Risk:** the job's code runs **on your machine**. On a public repo anyone can open a PR → never run PR workflows on self-hosted runners.
- Prefer ephemeral runners (a fresh VM/container per job) and the minimum access on that machine.

Debugging a red pipeline

- Read the **first** red step's log — later failures are usually consequences.
- **Re-run jobs → Enable debug logging** (or secret `ACTIONS_STEP_DEBUG` = `true`) for verbose logs.
- Run the same commands locally — the step is "just" a shell command.
- **actionlint** catches YAML and expression mistakes before you push.
- **act** runs workflows locally in Docker — handy, but not identical to GitHub's runners.

When it doesn’t work

Symptom	Likely cause
Workflow never starts	Wrong folder, YAML error, or the event filter doesn’t match
`write_package` denied	Missing `permissions: packages: write`
`repository name must be lowercase`	Capitals in user/repo used as image name
Check not offered in the ruleset	It hasn’t run yet, or the job name differs
Secret is empty	PR from a fork, or the job lacks `environment:`
Job skipped	A job in `needs:` failed, or `if:` was false
Deploy hangs	Waiting for **your** approval in the environment

Block 8

Today’s practice

Today’s practice — let the pipeline ship it

1. Repo `lab11-cicd` with the app, tests and Dockerfile
2. CI on every PR: lint, tests on 3.11 + 3.12, gitleaks
3. Break a test → red PR → fix → green
4. Ruleset on `main` : checks required, no direct pushes
5. Merge → image built, **Trivy-gated**, pushed to GHCR
6. Tag `v1.0.0` → image tag `1.0.0`

```
PR #1  ci
  ✓ lint
  ✗ test (3.11)   test_greet
  ✗ test (3.12)   test_greet
  ✓ secrets-scan
  - image (skipped)
```

Stretch: a `deploy` workflow with a `production` approval, then a rollback to the previous tag.

By the end of this lesson, you should be able to

- ☐ Explain CI, continuous delivery and continuous deployment
- ☐ Read and write a workflow: events, jobs, steps, matrix, `needs`, `if`
- ☐ Build, scan and push an image to GHCR with meaningful tags
- ☐ Add security gates and keep tokens least-privilege
- ☐ Make branch protection enforce your checks
- ☐ Deploy with an approval and roll back to an older tag

Before next lecture

- ☐ Finish today's practice if you didn't wrap it up in session
- ☐ Add the same CI to your final project repository (`css4003-final-project` — it already has a minimal `ci.yml`)
- ☐ Install Terraform (`terraform -version`)

Next week the infrastructure itself becomes code, reviewed and applied the same way.

Next lecture

Infrastructure as Code with Terraform — describe servers, networks and storage in code, review them in a PR, and apply them from a pipeline.